UNIVERSIDAD PRIVADA ANTENOR ORREGO

FACULTAD DE INGENIERÍA

ESCUELA PROFESIONAL DE INGENIERÍA DE COMPUTACIÓN Y SISTEMAS

Sistema de detección de correos electrónicos de phishing mediante aprendizaje automático y procesamiento de lenguaje natural

Proyecto de Aprendizaje Estadístico

Curso: Aprendizaje Estadístico

NRC: 4995

Integrantes:

Eustaquio Ávila, Joel Antony
Marquez Rodriguez, Franco Guillermo
Velásquez Revilla, Favio Adriano

Docente: Hernan Sagastegui Chigne

Semestre: 2026-II

Trujillo – Perú

I. INTRODUCCIÓN

1.1 Título del Proyecto

Sistema de detección de correos electrónicos de phishing mediante aprendizaje automático y procesamiento de lenguaje natural.

1.2 Antecedentes

El phishing es un tipo de ataque en el que una persona envía correos que parecen venir de un banco, una empresa o un servicio conocido, con el fin de robar contraseñas, datos bancarios o dinero. Es uno de los delitos informáticos más frecuentes. En 2025 fue el delito más denunciado ante el Internet Crime Complaint Center (IC3) del FBI, con 191 561 denuncias y pérdidas de 215,8 millones de dólares (FBI, 2026). Además, el 16 % de las filtraciones de datos en empresas empezó con un correo de phishing, con un costo promedio de 4,8 millones de dólares por filtración (IBM, 2025).

La detección automática de este tipo de correos se estudia desde hace varios años. Los trabajos que tomamos como base son:

Antecedente 1. Sahami et al. (1998). Crearon uno de los primeros filtros de correo no deseado basados en estadística. Usaron un clasificador Naive Bayes que calcula la probabilidad de que un mensaje sea no deseado según las palabras que contiene. Mostraron que las palabras de un correo sirven para clasificarlo de forma automática.

Antecedente 2. Abu-Nimeh et al. (2007). Compararon seis técnicas de aprendizaje automático para detectar correos de phishing, entre ellas la regresión logística, las máquinas de soporte vectorial, random forest y redes neuronales. Encontraron que el desempeño de cada técnica cambia según la métrica que se use, por ejemplo la tasa de error o la de falsos positivos. Por eso es importante definir desde el inicio qué métricas se van a evaluar.

Antecedente 3. Fette et al. (2007). Desarrollaron PILFER, un clasificador que usa características de la estructura del correo, como enlaces con direcciones IP o el uso de HTML. Su sistema detectó más del 96 % del phishing con muy pocos falsos positivos. Esto muestra que las características estructurales ayudan a mejorar la detección.

Antecedente 4. Bergholz et al. (2010). Propusieron nuevas características para filtrar phishing a partir del contenido y la estructura del correo, y probaron su método con correos etiquetados como spam, phishing y legítimos. Su trabajo muestra que el phishing y el spam son problemas distintos, algo que tomamos en cuenta como limitación de nuestro data-set.

Antecedente 5. Almomani et al. (2013). Hicieron una revisión de las técnicas de filtrado de correos de phishing, sobre todo las basadas en aprendizaje automático. Señalan que el phishing es un tipo especial de correo no deseado, porque busca suplantar a entidades reales para robar información.

Antecedente 6. Salloum et al. (2022). En una revisión sobre el uso del procesamiento de lenguaje natural (PLN) para detectar phishing, concluyeron que lo más importante es cómo se extraen las características del texto y que Python es la herramienta más usada. Esto respalda nuestro enfoque con TF-IDF y Python.

Antecedente 7. Al-Subaiey et al. (2024). Desarrollaron una plataforma web para detectar correos de phishing con aprendizaje automático, que alcanzó un F1-score de 0,99. Es un ejemplo de cómo un modelo de este tipo puede llevarse a una aplicación real.

Con base en estos trabajos, en este proyecto se desarrolla un sistema que analiza el texto de un correo y decide si es legítimo o phishing. Para esto se usan técnicas de PLN, TF-IDF, características estructurales del correo y un modelo de regresión logística.

1.3 Problema a resolver

Las personas reciben muchos correos al día y no siempre pueden reconocer cuáles son peligrosos. Revisarlos uno por uno toma tiempo y depende de que el usuario sepa identificar las señales de un phishing.

El problema consiste en desarrollar un sistema que clasifique automáticamente un correo en una de dos clases:

- **0 – Safe Email:** correo legítimo.
- **1 – Phishing Email:** correo de phishing.

Es decir, se busca aprender una función f: X → {0, 1}, donde X es la representación numérica de los correos. El modelo calcula la probabilidad de que un correo sea phishing y, si es mayor o igual a 0,5, lo clasifica como phishing.

El problema tiene tres dificultades:

- **Evasión:** los atacantes cambian la forma de escribir sus correos para que los filtros no los detecten.
- **Desbalance de clases:** en el data-set hay más correos legítimos (62,6 %) que de phishing (37,4 %).
- **Costo de los errores:** si el sistema deja pasar un phishing, el usuario puede ser estafado; si bloquea un correo legítimo, el usuario pierde información importante. Por eso los dos errores deben ser bajos.

1.4 Objetivos

Objetivo general

Desarrollar un sistema de aprendizaje automático que detecte correos de phishing a partir de su texto y de algunas características de su estructura.

Objetivos específicos

1. Preparar y limpiar un data-set de correos etiquetados.
2. Normalizar el texto de los correos y convertirlo en números con TF-IDF.
3. Calcular características estructurales de cada correo.
4. Entrenar un modelo de regresión logística.
5. Evaluar el modelo con métricas de clasificación.
6. Implementar una función que clasifique correos nuevos.

Metas de desempeño

El modelo se considera exitoso si cumple estas metas en el conjunto de prueba:

Tabla 1. Metas de desempeño del modelo

Métrica	Qué mide	Meta
Accuracy	Porcentaje de correos bien clasificados	> 90 %
Precision	De los correos marcados como phishing, cuántos sí lo eran	> 90 %
Recall	De los correos de phishing, cuántos se detectaron	> 90 %
F1-score	Equilibrio entre precision y recall	> 0,90
Tasa de falsos positivos (FPR)	Porcentaje de correos legítimos marcados como phishing	< 5 %

II. REQUERIMIENTOS

2.1 Definición del Dominio

El proyecto pertenece al área de la **ciberseguridad**, específicamente a la detección automática de correos de phishing.

Desde el punto de vista del aprendizaje estadístico, es un problema de **clasificación binaria supervisada**: binaria porque solo hay dos clases (Safe y Phishing), y supervisada porque cada correo del data-set ya tiene su etiqueta.

Alcance: el sistema trabaja con correos en inglés, que es el idioma del data-set, y solo analiza el texto del mensaje, porque el data-set no incluye remitente, destinatario ni fecha. Por eso las características estructurales se calculan a partir del mismo texto.

2.2 Determinación de Requisitos (Requerimientos)

Requisitos funcionales

El sistema debe:

- RF-01: cargar el data-set de correos etiquetados (archivo CSV).
- RF-02: limpiar los datos y normalizar el texto.
- RF-03: convertir el texto en números con TF-IDF y calcular las características estructurales.
- RF-04: dividir los datos en entrenamiento y prueba, y aplicar validación cruzada.
- RF-05: entrenar el modelo de regresión logística.
- RF-06: evaluar el modelo con las metas definidas.
- RF-07: clasificar un correo nuevo como legítimo o phishing.

Requisitos no funcionales

El sistema debe:

- RNF-01: dar resultados rápidos.
- RNF-02: ser reproducible, usando una semilla fija (random_state = 42).
- RNF-03: evitar la fuga de datos entre entrenamiento y prueba.
- RNF-04: tener un código ordenado y fácil de entender.

Herramientas

Tabla 2. Herramientas usadas en el proyecto

Herramienta	Uso en el proyecto
Python y Google Colab	Lenguaje y entorno donde se desarrolla el sistema
pandas y NumPy	Carga, limpieza y manejo de los datos
scikit-learn	TF-IDF, escalado, pipeline, regresión logística, validación cruzada y métricas
NLTK	Stemming con el algoritmo de Porter
matplotlib	Gráficos de los datos y de los resultados
WEKA 3.9.7	Revisión del data-set limpio (distribución de clases y características)

III. PLANTEAMIENTO DEL DATA-SET (APRENDIZAJE SUPERVISADO), PRE-PROCESAMIENTO Y NORMALIZACIÓN

Todo el desarrollo se hizo en Python con Google Colab. El cuaderno completo se puede ver en el siguiente enlace:

https://colab.research.google.com/drive/1EVCmeimmvzYhb1KEwd6lxX0e0ZgqGwuv

3.1 Medidas, Datos, Bases de Datos, Elaboración de Data-Set, y Características (Features)

Se usó el data-set público *Phishing Email Detection*, publicado en Kaggle (subhajournal, 2023). Tiene 18 650 correos en inglés y 3 columnas. Es un problema de aprendizaje supervisado, porque cada correo ya viene con su etiqueta.

Tabla 3. Columnas del data-set original

Columna	Descripción	Uso
Unnamed: 0	Número de fila	Se elimina
Email Text	Texto completo del correo	Entrada del modelo (X)
Email Type	Safe Email o Phishing Email	Salida (y): Safe = 0, Phishing = 1

Al revisar el archivo encontramos 16 correos sin texto. Antes de la limpieza había 11 322 correos legítimos (60,7 %) y 7 328 de phishing (39,3 %).

Como medidas, además del texto, a cada correo se le calcularon 6 características estructurales: número de enlaces, cantidad de caracteres, cantidad de palabras, proporción de mayúsculas, número de signos de exclamación y número de palabras de urgencia. Se explican en el punto 3.2.

3.2 Normalización y/o Filtrado de Datos

a) Limpieza de registros

Primero se eliminó la columna de índice y luego los correos que no servían para entrenar:

Tabla 4. Limpieza de registros por etapa

Etapa	Correos que quedan	Correos eliminados
Data-set original	18 650	–
Sin textos nulos	18 634	16
Sin textos vacíos	18 631	3
Sin duplicados	17 536	1 095

Los duplicados se quitaron para que un mismo correo no quede a la vez en entrenamiento y en prueba, porque eso haría que el modelo parezca mejor de lo que es. También se recortaron a 20 000 caracteres los textos demasiado largos (87 correos; uno tenía más de 17 millones de caracteres por un error del archivo).

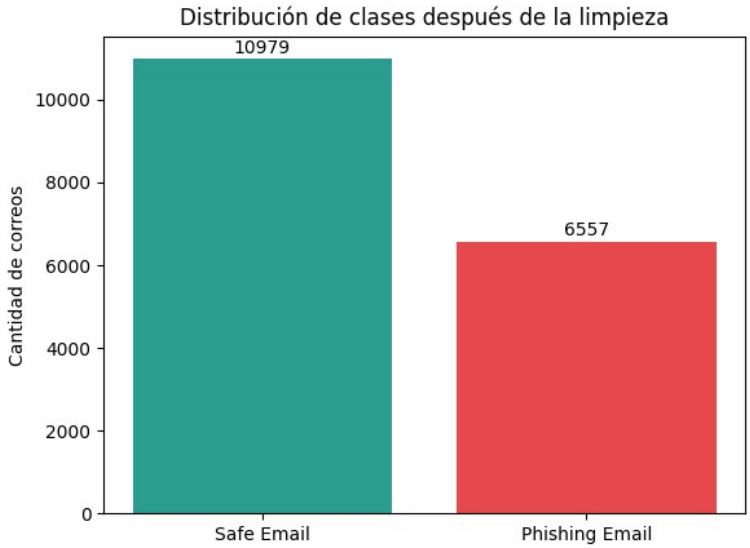


Figura 1. Distribución de clases después de la limpieza.

En la Figura 1 se ve que, después de la limpieza, quedaron 17 536 correos: 10 979 legítimos (62,6 %) y 6 557 de phishing (37,4 %). Hay más correos legítimos, pero el desbalance es moderado. Para compensarlo, en el modelo se usa el parámetro class_weight = "balanced" (sección IV).

b) Características estructurales

Además del texto, se calcularon 6 medidas de cada correo. La Tabla 5 muestra qué mide cada una y su promedio por clase:

Tabla 5. Características estructurales y su promedio por clase

Característica	Qué mide	Promedio Safe	Promedio Phishing
n_urls	Cantidad de enlaces	0,771	0,292
n_caracteres	Largo del correo en caracteres	1 848,7	1 641,6
n_palabras	Cantidad de palabras	350,1	311,6
prop_mayus	Proporción de letras en mayúscula	0,022	0,027
n_exclam	Cantidad de signos "!"	0,638	3,385
n_urgencia	Palabras de presión (urgent, verify, account, password, bank, login, entre otras)	0,352	1,081

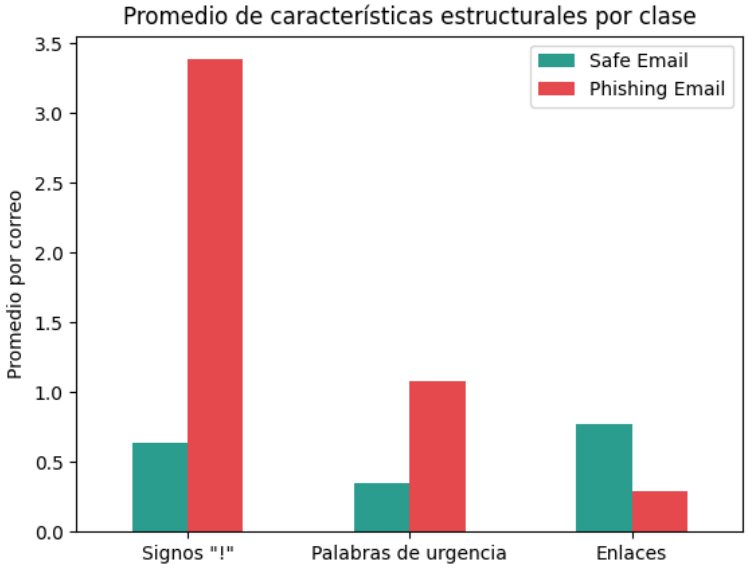


Figura 2. Promedio de características estructurales por clase.

En la Figura 2 se ve que los correos de phishing tienen unas 5 veces más signos de exclamación y unas 3 veces más palabras de urgencia que los legítimos. Esto tiene sentido, porque el phishing busca presionar al usuario para que actúe rápido. En cambio, los correos legítimos tienen más enlaces, porque muchos vienen de listas de correo que incluyen varias direcciones web.

c) Normalización del texto

Antes de convertir el texto en números, se normalizó con estos pasos:

1. Pasar todo a minúsculas.
2. Quitar las etiquetas HTML.
3. Reemplazar los enlaces por la palabra "url".
4. Reemplazar las direcciones de correo por la palabra "email".
5. Quitar números y signos.
6. Quitar las palabras vacías (stopwords), como "the" o "and", y las palabras de menos de 3 letras.
7. Aplicar stemming con el algoritmo de Porter (1980), que deja cada palabra en su raíz.

Por ejemplo, este es uno de los correos antes y después de la normalización:

Tabla 6. Ejemplo de un correo antes y después de la normalización

	Texto
Antes	the other side of * galicismos * * galicismo * is a spanish term which names the improper introduction of french words which are spanish sounding and thus very deceptive to the ear …
Después	galicismo galicismo spanish term name improp introduct french word spanish sound decept ear …

d) Conversión a números: TF-IDF y Min-Max

El texto normalizado se convirtió en números con **TF-IDF**, que le da más peso a una palabra que aparece mucho en un correo pero poco en los demás, porque esa palabra es la que lo distingue. Se usó la versión de scikit-learn con tf sublineal:

$$tf\text{-}idf(t, d) = [1 + \log tf(t, d)] \times [\ln((1 + n) / (1 + df(t))) + 1]$$

donde tf(t, d) es cuántas veces aparece el término t en el correo d, n es el número de correos y df(t) es en cuántos correos aparece t. Se usaron palabras sueltas y pares de palabras (bigramas), con un máximo de 5 000 términos que aparezcan en al menos 3 correos.

Las 6 características estructurales tienen escalas muy distintas (por ejemplo, n_caracteres llega a 20 000 y prop_mayus no pasa de 1). Por eso primero se les aplicó log(1 + x) para reducir los valores extremos y luego la normalización **Min-Max**, que las lleva al rango de 0 a 1:

$$x' = (x - mín) / (máx - mín)$$

El TF-IDF y el Min-Max se ajustan solo con los datos de entrenamiento, dentro de un pipeline (sección IV). Así se evita la fuga de datos hacia el conjunto de prueba.

e) Revisión del data-set en WEKA

Para revisar los datos limpios con otra herramienta, las 6 características y la clase se exportaron desde Colab en formato ARFF y se abrieron en WEKA 3.9.7. No se usó el CSV original porque sus textos son muy largos y tienen comas, comillas y saltos de línea que WEKA no lee bien, y porque no trae las características estructurales.

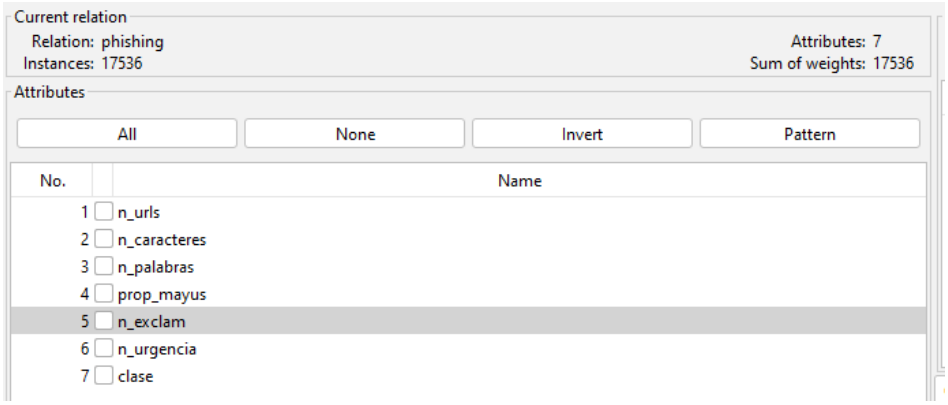


Figura 3. Data-set limpio cargado en WEKA (pestaña Preprocess).

En la Figura 3 se ve que WEKA cargó los 17 536 correos con 7 atributos: las 6 características estructurales y la clase. Esto coincide con el resultado de la limpieza en Colab.

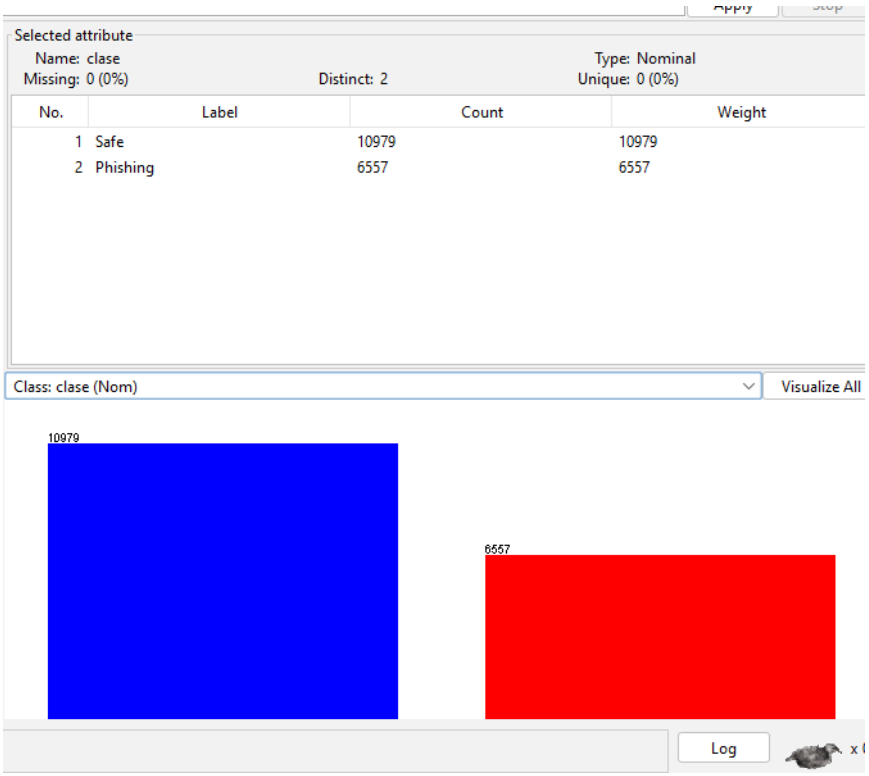


Figura 4. Distribución de la clase en WEKA.

En la Figura 4, WEKA muestra 10 979 correos Safe (azul) y 6 557 Phishing (rojo), sin valores faltantes. Es la misma distribución de la Figura 1.

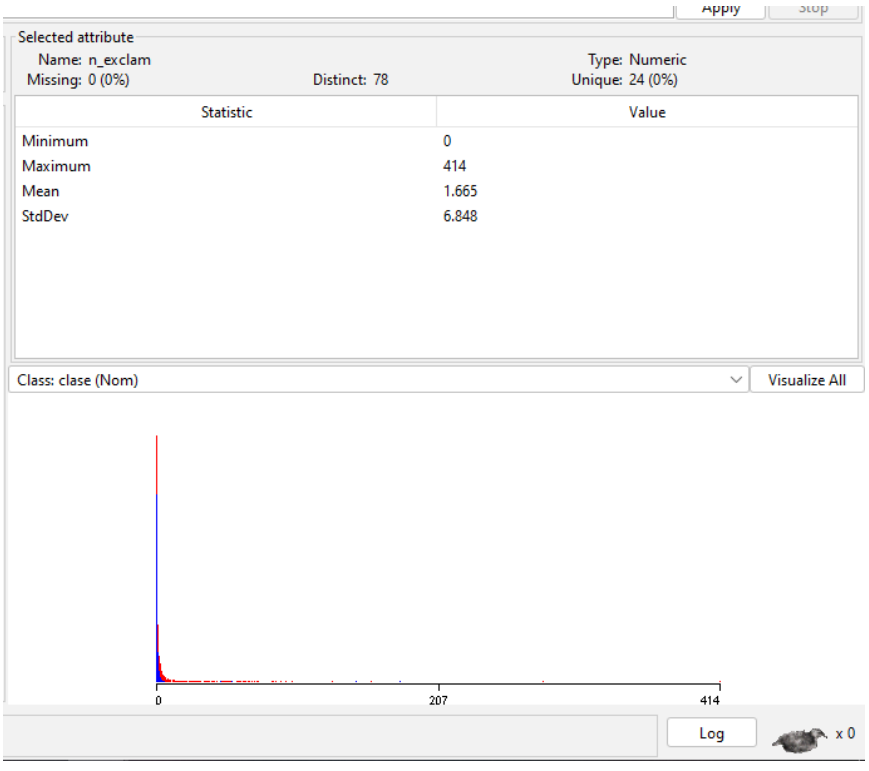


Figura 5. Estadísticas e histograma de n_exclam en WEKA.

En la Figura 5 se ve que la mayoría de correos tiene pocos signos de exclamación (media de 1,665), pero hay casos extremos de hasta 414. En las barras cercanas a cero aparece bastante rojo, lo que confirma que el phishing usa más signos de exclamación.

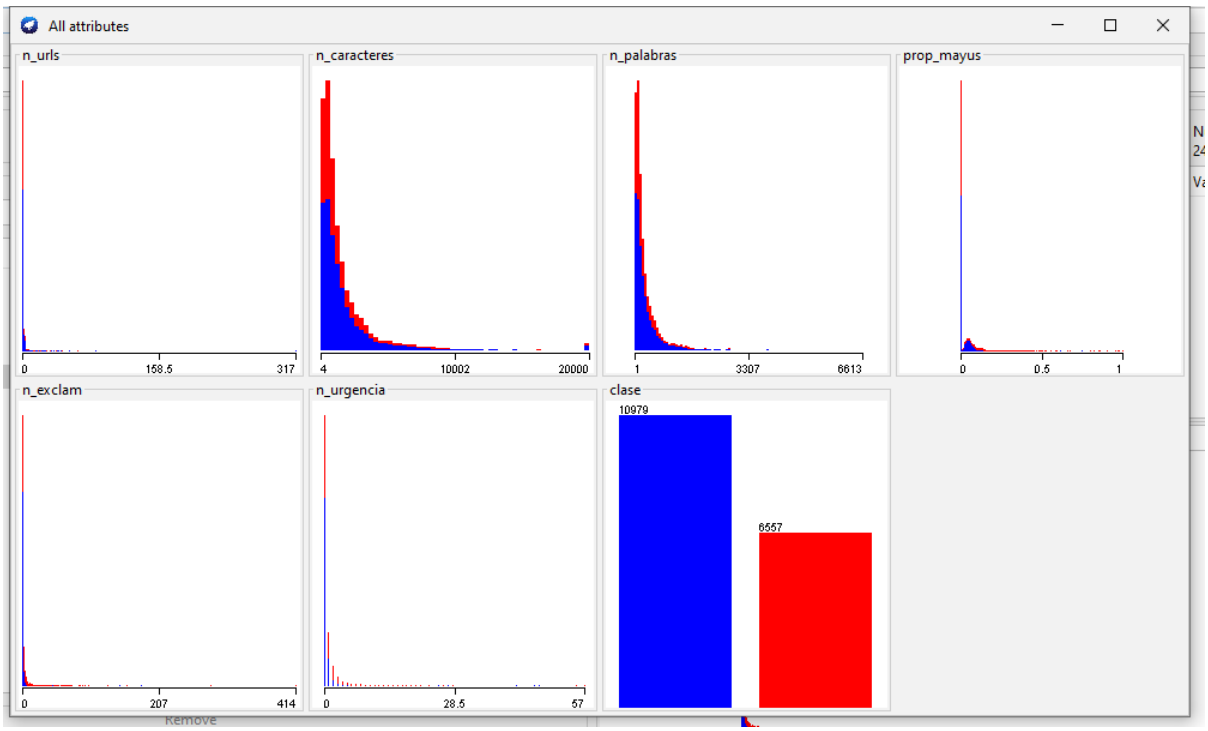


Figura 6. Histogramas de todos los atributos en WEKA (azul = Safe, rojo = Phishing).

En la Figura 6 se ve que todas las características se concentran en valores bajos y tienen pocos casos muy altos. Por eso se aplicó el logaritmo y el Min-Max antes del modelo. También se nota que en n_caracteres y n_palabras hay proporcionalmente más rojo en los valores pequeños, es decir, los correos de phishing suelen ser un poco más cortos (en promedio 1 641,6 caracteres frente a 1 848,7 de los legítimos).

Por último, en WEKA se probó la regresión logística (Classify → functions → Logistic) usando solo las 6 características estructurales, con validación cruzada de 10 pliegues:

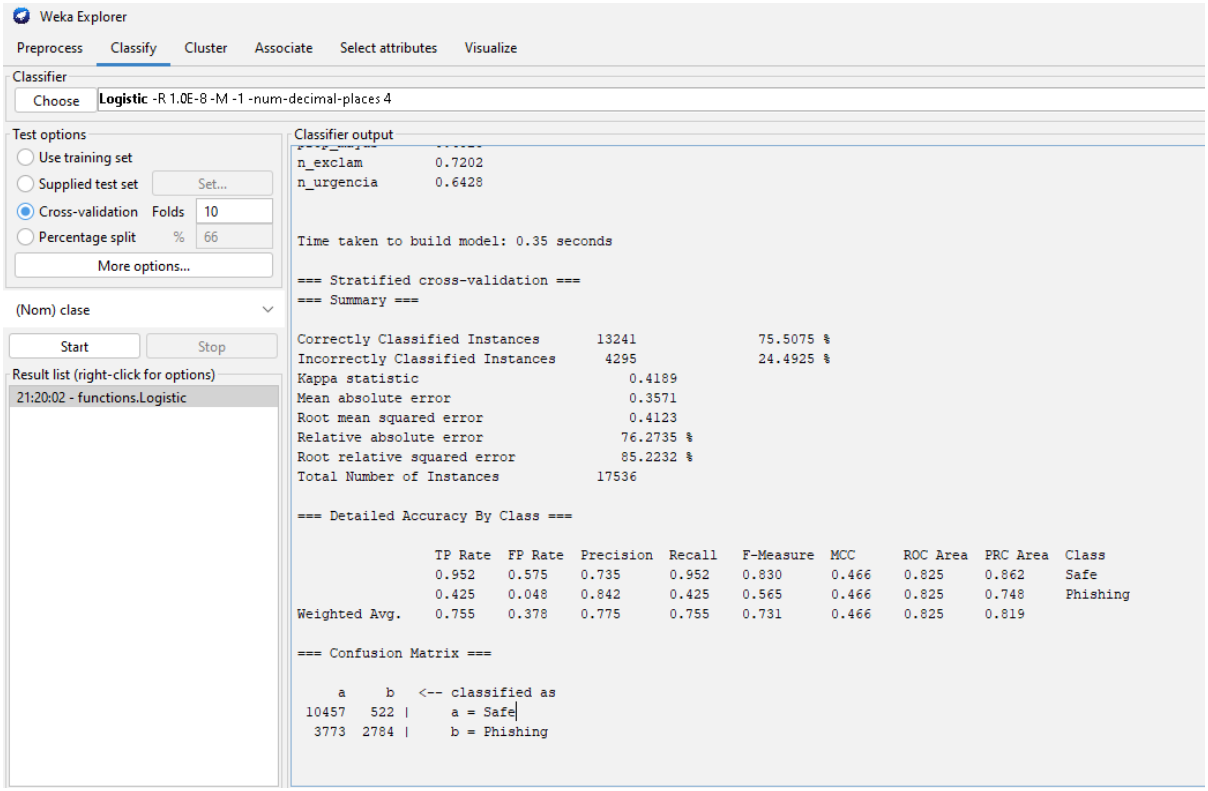


Figura 7. Regresión logística en WEKA solo con las características estructurales.

Como se ve en la Figura 7, el modelo acertó el 75,5 % de los correos, pero solo detectó el 42,5 % del phishing: de 6 557 correos de phishing, 3 773 pasaron como legítimos. Esto pasa porque lo que más delata a un phishing son sus palabras, y estas no están en el archivo ARFF. Por eso el modelo final se entrenó en Colab con el texto (TF-IDF) y las características estructurales juntas.

3.3 Planteamiento de los Data-Sets de Entrenamiento, Pruebas y Validación

Los 17 536 correos se dividieron de forma estratificada, es decir, manteniendo el mismo porcentaje de phishing en cada parte, y con una semilla fija (random_state = 42):

Tabla 7. División del data-set

Conjunto	Porcentaje	Correos	Phishing	Uso
Entrenamiento (training)	70 %	12 275	4 590 (37,4 %)	Entrenar el modelo
Prueba (test)	30 %	5 261	1 967 (37,4 %)	Evaluación final, una sola vez
Validación (cross-validation)	10 pliegues	≈ 1 227 por pliegue	≈ 37,4 %	Medir la estabilidad del modelo

- **Data-Set de entrenamiento (training):** 12 275 correos (70 %). Con ellos se ajustan el TF-IDF, el Min-Max y la regresión logística.
- **Data-Set de pruebas (test):** 5 261 correos (30 %). No se usan para nada durante el entrenamiento; solo sirven para la evaluación final.
- **Data-Set de validación por Cross-Validation:** dentro del entrenamiento se usó validación cruzada estratificada de 10 pliegues. El entrenamiento se divide en 10 partes de unos 1 227 correos; el modelo se entrena 10 veces con 9 partes y se valida con la parte que queda. El promedio de las 10 rondas muestra si el modelo es estable.

IV. APRENDIZAJE

4.1 Planteamiento del Modelo de Aprendizaje (Machine Learning supervisado)

El modelo es de **aprendizaje supervisado**, porque aprende de pares (x_i, y_i). Cada x_i es un correo convertido en 5 006 números (5 000 valores de TF-IDF y 6 características estructurales) e y_i es su etiqueta: 0 si es legítimo y 1 si es phishing.

Como modelo se eligió la **regresión logística**, por tres razones:

- Da directamente la probabilidad de que un correo sea phishing, que se compara con el umbral de 0,5.
- Funciona bien con datos de texto, que tienen muchas columnas y la mayoría con valor cero.
- Es fácil de interpretar: cada término tiene un peso que indica si empuja hacia phishing (positivo) o hacia legítimo (negativo).

El modelo calcula la probabilidad de phishing con la función sigmoide:

$$P(y = 1 \mid x) = 1 / (1 + e^{-(w \cdot x + b)})$$

donde w son los pesos que aprende el modelo y b es el sesgo. Si el resultado es mayor o igual a 0,5, el correo se clasifica como phishing. El modelo usa regularización L2, que evita que los pesos crezcan demasiado y así reduce el sobreajuste.

4.2 Desarrollo e Implementación del Modelo

El modelo se implementó en Python con scikit-learn, dentro de un **pipeline** que junta todos los pasos desde el correo hasta la predicción:

Correo → Normalización del texto → TF-IDF + características estructurales (log + Min-Max) → Regresión logística → Clase y probabilidad

La implementación tiene tres partes:

- **Preprocesador (ColumnTransformer):** aplica TF-IDF al texto normalizado y log + Min-Max a las 6 características, y junta todo en una matriz de 5 006 columnas.
- **Pipeline (función crear_pipeline):** une el preprocesador con la regresión logística. Así, en cada pliegue de la validación cruzada, el preprocesador se ajusta solo con los datos de entrenamiento de ese pliegue y no hay fuga de datos.
- **Función de clasificación (clasificar_correo):** recibe el texto de un correo nuevo, le aplica los mismos pasos y devuelve la clase y la probabilidad de phishing.

El modelo se configuró como LogisticRegression(C = 1.0, max_iter = 1000, class_weight = "balanced", random_state = 42). El parámetro **class_weight = "balanced"** le da más peso a los errores en la clase phishing, que es la minoritaria, para compensar el desbalance de 62,6 % y 37,4 %.

```
[11]  def crear_pipeline():
          preprocesador = ColumnTransformer([
              ("tfidf", TfidfVectorizer(ngram_range=(1, 2), max_features=5000, min_df=3,
                                        sublinear_tf=True), "texto_limpio"),
              ("estruct", Pipeline([("log", FunctionTransformer(np.log1p)),
                                    ("minmax", MinMaxScaler())]), COLS),
          ])
          modelo = LogisticRegression(C=1.0, max_iter=1000, class_weight="balanced",
                                      random_state=SEMILLA)
          return Pipeline([("preprocesador", preprocesador), ("modelo", modelo)])

      def clasificar_correo(texto, pipeline):
          fila = pd.DataFrame([{"texto_limpio": limpiar_texto(texto), **estructurales(texto)}])
          p = pipeline.predict_proba(fila)[0, 1]          # probabilidad de phishing
          clase = "PHISHING" if p >= 0.5 else "LEGÍTIMO"
          return clase, round(float(p), 4)
```

Figura 8. Código del pipeline y de la función de clasificación en Google Colab.

En la Figura 8 se ve el código del pipeline. La función crear_pipeline arma el preprocesador y el modelo juntos, y la función clasificar_correo permite usar el modelo entrenado con cualquier correo nuevo.

V. COMPROBACIÓN

5.1 Entrenamiento del Modelo

El entrenamiento se hizo en tres pasos:

1. Validación cruzada de 10 pliegues con el conjunto de entrenamiento, para ver si el modelo es estable.
2. Entrenamiento final del modelo con los 12 275 correos de entrenamiento.
3. Aplicación del modelo al conjunto de prueba (5 261 correos).

Validación cruzada

Tabla 8. Resultados de la validación cruzada de 10 pliegues

Pliegue	Accuracy	Precision	Recall	F1-score	ROC-AUC
1	0,9756	0,9694	0,9651	0,9672	0,9960
2	0,9723	0,9590	0,9673	0,9631	0,9960
3	0,9699	0,9607	0,9586	0,9597	0,9959
4	0,9666	0,9485	0,9630	0,9557	0,9923
5	0,9707	0,9415	0,9826	0,9616	0,9948
6	0,9698	0,9587	0,9608	0,9597	0,9957
7	0,9772	0,9634	0,9760	0,9697	0,9958
8	0,9747	0,9612	0,9717	0,9664	0,9970
9	0,9788	0,9520	0,9935	0,9723	0,9965
10	0,9731	0,9651	0,9630	0,9640	0,9971
Promedio	**0,9729**	**0,9579**	**0,9702**	**0,9639**	**0,9957**

En la Tabla 8 se ve que el F1-score va de 0,9557 (pliegue 4) a 0,9723 (pliegue 9), con un promedio de 0,9639. Los resultados son muy parecidos en los 10 pliegues, lo que indica que el modelo es estable y no depende de qué correos le toquen. Además, en todos los pliegues las métricas superan el 90 %.

Aplicación al Modelo: uso del Data-Set de Entrenamiento y de Prueba

Después de la validación cruzada, el modelo se entrenó con los 12 275 correos de entrenamiento, lo que tomó 8,6 segundos. Luego se aplicó a los 5 261 correos de prueba, que el modelo no había visto antes. Los resultados de esta prueba se evalúan en la sección VI.

VI. EVALUACIÓN

6.1 Métricas y Evaluación del Modelo

Con los 5 261 correos de prueba se calcularon las métricas definidas en el punto 1.4 y se compararon con las metas:

Tabla 9. Resultados en el conjunto de prueba comparados con las metas

Métrica	Resultado	Meta	¿Cumple?
Accuracy	97,66 %	> 90 %	Sí
Precision	96,33 %	> 90 %	Sí
Recall	97,46 %	> 90 %	Sí
F1-score	0,9689	> 0,90	Sí
Tasa de falsos positivos	2,22 %	< 5 %	Sí

El modelo cumple las cinco metas. Además, los resultados en la prueba (F1 de 0,9689) son casi iguales a los de la validación cruzada (F1 de 0,9639), lo que indica que el modelo funciona bien con correos que no vio durante el entrenamiento.

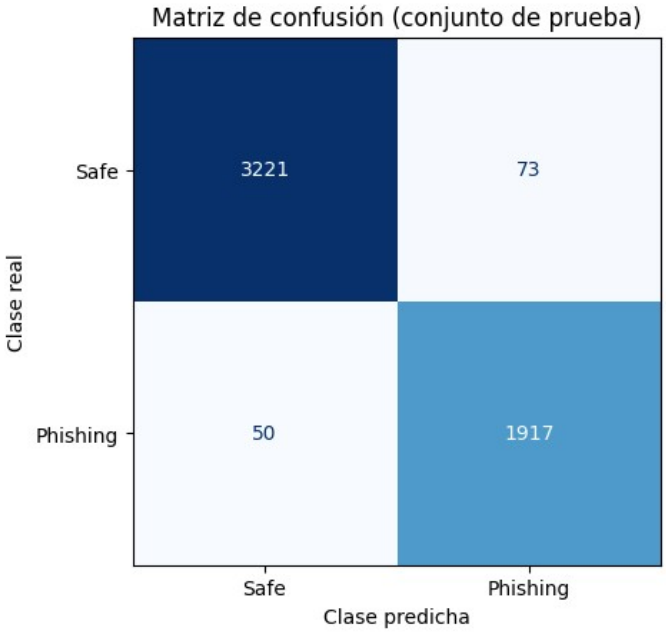


Figura 9. Matriz de confusión en el conjunto de prueba.

En la Figura 9, de los 3 294 correos legítimos, el modelo clasificó bien 3 221 y marcó 73 como phishing por error (falsos positivos). De los 1 967 correos de phishing, detectó 1 917 y solo dejó pasar 50 (falsos negativos). Es decir, se equivoca en apenas 123 de 5 261 correos.

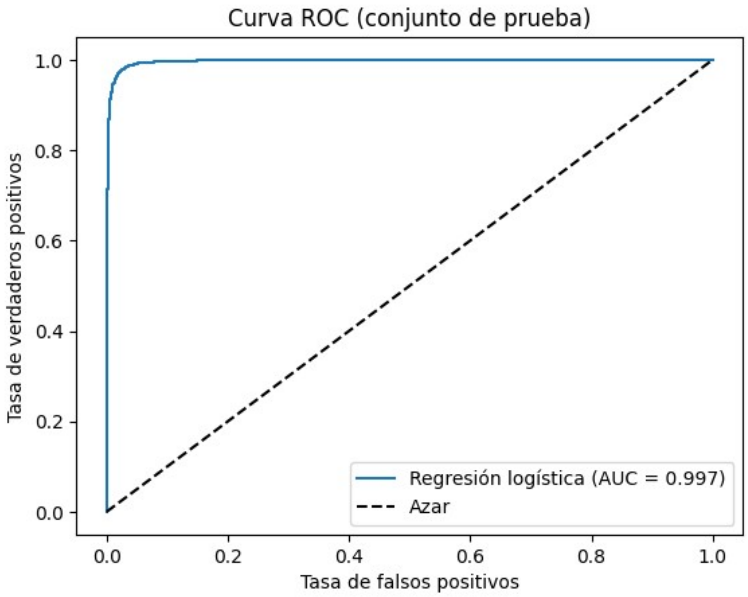


Figura 10. Curva ROC en el conjunto de prueba.

En la Figura 10, la curva ROC sube casi en vertical y se pega a la esquina superior izquierda, muy lejos de la línea del azar. El área bajo la curva (AUC) es 0,997, muy cerca de 1. Esto significa que el modelo separa muy bien los correos legítimos de los de phishing.

Si comparamos con la prueba en WEKA usando solo las características estructurales (Figura 7), el recall del phishing pasó de 42,5 % a 97,5 % al agregar el texto con TF-IDF. Esto confirma que las palabras del correo son la información más importante para detectar el phishing.

6.2 Evaluación de desempeño del Sistema de IA del Proyecto

En el punto 6.1 se evaluó el modelo con métricas. En este punto se evalúa el sistema completo: qué tan rápido responde y cómo se comporta con correos nuevos.

Tabla 10. Tiempo de respuesta del sistema

Tarea	Tiempo
Entrenar el modelo (12 275 correos)	8,6 s
Clasificar el conjunto de prueba (5 261 correos)	1,70 s
Clasificar un correo nuevo	7,7 ms

El sistema clasifica un correo nuevo en menos de 8 milisegundos, así que puede usarse en tiempo real. Luego se probó con seis correos escritos por nosotros, que no están en el data-set:

```
Aciertos: 5 de 6
```

	Correo	Esperado	Predicho	P(phishing)	¿Acierta?
0	Hi John, attached is the report for tomorrow's...	LEGÍTIMO	LEGÍTIMO	0.0180	Sí
1	Reminder: the team lunch was moved to Friday a...	LEGÍTIMO	LEGÍTIMO	0.1379	Sí
2	Hello professor, I am sending you the homework...	LEGÍTIMO	PHISHING	0.7833	No
3	URGENT: Your account has been suspended! Verif...	PHISHING	PHISHING	0.7797	Sí
4	Congratulations!!! You have won $1,000,000. Cl...	PHISHING	PHISHING	0.7846	Sí
5	Dear customer, we detected unusual activity in...	PHISHING	PHISHING	0.8501	Sí

Figura 11. Resultado de la prueba del sistema con correos nuevos en Google Colab.

En la Figura 11 se ve que el sistema acertó 5 de los 6 correos. Detectó los 3 de phishing, con probabilidades de 0,78 a 0,85, y reconoció 2 de los 3 legítimos (0,018 y 0,138). Falló con el correo de un estudiante a su profesor, que marcó como phishing con 0,78. Esto pasa porque los correos legítimos del data-set son sobre todo de empresas y listas de correo, y el modelo no vio mensajes de ese tipo.

En general, el sistema cumple las metas y responde rápido. Su principal limitación son los correos legítimos que no se parecen a los del data-set.

VII. DESPLIEGUE (DEPLOY) DEL SISTEMA DEL PROYECTO DE APRENDIZAJE ESTADÍSTICO

Para que el sistema se pueda usar fuera del Colab, el modelo entrenado se guardó en un archivo y se publicó junto con una aplicación web. El código y la documentación están en GitHub, y la aplicación está publicada en Streamlit Community Cloud.

7.1 Publicación del Proyecto y del Sistema en GITHUB

El proyecto se publicó en un repositorio público de GitHub: https://github.com/ByArk3/phishing-detector-ML. El repositorio se organizó en tres partes, que corresponden a las pestañas que pide la plantilla: documentación, código, y ejecución y pruebas.

7.1.1 Pestaña de Documentación del Proyecto

La documentación está en el archivo README.md, que GitHub muestra en la página principal del repositorio. Tiene el problema, el data-set (con el enlace a Kaggle), los pasos del modelo, los resultados principales y los enlaces al informe y al Colab. En la carpeta docs/ están el informe del proyecto en PDF y la estructura del proyecto que pide el curso.

7.1.2 Pestaña de Código del Sistema

El código del sistema está en la raíz del repositorio y en la carpeta notebooks/:

Tabla 11. Archivos de código del repositorio

Archivo	Qué hace
app.py	Aplicación web en Streamlit. Recibe el texto del correo y muestra si es phishing o legítimo, con su probabilidad.
procesamiento.py	Funciones de limpieza del texto y de las 6 características, iguales a las del Colab.
modelo_phishing.joblib	Modelo ya entrenado en el Colab (TF-IDF + características + regresión logística).
requirements.txt	Librerías con las mismas versiones del Colab.
notebooks/Proyecto_Phishing_Colab.ipynb	Colab completo, desde la carga del data-set hasta la creación de los archivos de la app.
pruebas/prueba_correos_nuevos.py	Prueba el modelo guardado con 6 correos nuevos.

La app no vuelve a entrenar el modelo: carga modelo_phishing.joblib, que se generó en la sección VII del Colab. Para que cargue sin errores, requirements.txt usa las mismas versiones de librerías del Colab (por ejemplo, scikit-learn 1.6.1).

7.1.3 Pestaña de Ejecución y Pruebas del Sistema

En el README se explica cómo ejecutar el sistema en una computadora con Python 3.13:

```
pip install -r requirements.txt
streamlit run app.py
```

También se incluye el script pruebas/prueba_correos_nuevos.py, que carga el modelo guardado y clasifica los 6 correos nuevos del punto 6.2. Da las mismas probabilidades que el Colab (Figura 11): acierta 5 de 6 correos y vuelve a fallar con el correo al profesor. Esto confirma que el modelo publicado es el mismo que se evaluó en la sección VI.

7.2 Deploy del APP o Web del Sistema de Clasificación

La aplicación web se hizo con Streamlit y se publicó gratis en Streamlit Community Cloud, que la toma directamente del repositorio de GitHub. Se puede usar desde cualquier navegador en: https://detector-phishing-ml.streamlit.app.



Figura 12. Pantalla principal de la aplicación web.

En la Figura 12 se ve la pantalla principal. Arriba muestra los resultados del modelo (97,7 % de aciertos, F1 de 0,9689, AUC de 0,997 y 17 536 correos). En la pestaña Analizar, el usuario pega el texto de un correo, o elige uno de los seis correos de ejemplo, y presiona Analizar para ver si es phishing o legítimo y con qué probabilidad.



Figura 13. Pestaña “Sobre el modelo”: data-set, modelo y matriz de confusión.

En la Figura 13, la pestaña “Sobre el modelo” resume el data-set y el modelo, y muestra la matriz de confusión del conjunto de prueba: 3 221 verdaderos negativos, 73 falsos positivos, 50 falsos negativos y 1 917 verdaderos positivos. Son los mismos valores de la Figura 9.



Figura 14. Métricas y palabras más importantes del modelo en la aplicación.

En la Figura 14 se ven la precision (0,9633), el recall (0,9746) y la tasa de falsos positivos (0,0222), además de las palabras con más peso en la regresión logística. Palabras como “click”, “money” o “free” empujan hacia phishing, y palabras como “enron” o “linguist” hacia legítimo, porque muchos correos legítimos del data-set vienen de la empresa Enron y de una lista de lingüística. Esto explica por qué un correo legítimo distinto a esos, como el del profesor, puede salir como phishing.

REFERENCIAS

1. Abu-Nimeh, S., Nappa, D., Wang, X., & Nair, S. (2007). A comparison of machine learning techniques for phishing detection. En Proceedings of the Anti-Phishing Working Groups 2nd Annual eCrime Researchers Summit (pp. 60–69). ACM.
2. Al-Subaiey, A., Al-Thani, M., Alam, N. A., Antora, K. F., Khandakar, A., & Zaman, S. M. A. U. (2024). Novel interpretable and robust web-based AI platform for phishing email detection. arXiv. https://doi.org/10.48550/arXiv.2405.11619
3. Almomani, A., Gupta, B. B., Atawneh, S., Meulenberg, A., & Almomani, E. (2013). A survey of phishing email filtering techniques. IEEE Communications Surveys & Tutorials, 15(4), 2070–2090. https://doi.org/10.1109/SURV.2013.030713.00020
4. Bergholz, A., De Beer, J., Glahn, S., Moens, M.-F., Paaß, G., & Strobel, S. (2010). New filtering approaches for phishing email. Journal of Computer Security, 18(1), 7–35. https://doi.org/10.3233/JCS-2010-0371

5. Federal Bureau of Investigation. (2026). 2025 Internet Crime Report. Internet Crime Complaint Center (IC3). https://www.ic3.gov/AnnualReport/Reports/2025_IC3Report.pdf

6. Fette, I., Sadeh, N., & Tomasic, A. (2007). Learning to detect phishing emails. En Proceedings of the 16th International Conference on World Wide Web (pp. 649–656). ACM.

7. IBM. (2025). Cost of a Data Breach Report 2025. IBM Security; Ponemon Institute. https://www.ibm.com/reports/data-breach

8. Porter, M. F. (1980). An algorithm for suffix stripping. Program, 14(3), 130–137.

9. Sahami, M., Dumais, S., Heckerman, D., & Horvitz, E. (1998). A Bayesian approach to filtering junk e-mail. En Learning for Text Categorization: Papers from the 1998 Workshop (Technical Report WS-98-05). AAAI Press.

10. Salloum, S., Gaber, T., Vadera, S., & Shaalan, K. (2022). A systematic literature review on phishing email detection using natural language processing techniques. IEEE Access, 10, 65703–65727. https://doi.org/10.1109/ACCESS.2022.3183083

11. subhajournal. (2023). Phishing Email Detection [Data set]. Kaggle. https://www.kaggle.com/datasets/subhajournal/phishingemails